

Branch-and-Bound: Knights on the Chessboard 2

IV Operations Research — ST 2026

Swayam Virmani
(m12552861)

Shivam Lahoty
(m12550428)

Somya Khera
(m12550438)

Harman Kaur Bhambra
(m12552803)

1. Problem Definition

The **Knight Domination Problem** asks: given an $n \times n$ chessboard, place the minimum number of knights such that every square is either occupied by a knight or attacked by at least one knight. This is a combinatorial optimisation problem with applications in covering and domination problems in graph theory.

2. Integer Linear Program

Label each square $j \in \{0, \dots, n^2-1\}$ with a binary decision variable $x_j \in \{0, 1\}$, where $x_j = 1$ means a knight is placed on square j .

Let $A(j)$ denote the set of squares from which a knight can attack square j (i.e. the squares whose knight-move reaches j). Then the ILP is:

$$\min \sum_{j=0}^{n^2-1} x_j \quad (1)$$

$$\text{subject to } x_j + \sum_{i \in A(j)} x_i \geq 1 \quad \forall j \in \{0, \dots, n^2-1\} \quad (2)$$

$$x_j \in \{0, 1\} \quad \forall j \in \{0, \dots, n^2-1\} \quad (3)$$

Constraint (2) ensures every square j is threatened by at least one placed knight. Squares with $|A(j)| = 0$ (e.g. $n = 1$) make the problem infeasible. The LP relaxation replaces (3) with $0 \leq x_j \leq 1$ and is used as the bounding subproblem at every B&B node.

3. Implementation

3.1. Architecture

The solver is structured as five modules: `board.py` (geometry and attack map), `ilp_solver.py` (Gurobi ILP/LP interface), `bnb.py` (branch-and-bound engine), `heuristics.py` (upper bound), `lower_bounds.py` (lower bounds), and `utils.py` (I/O, timing, CSV export). The entry point is `main.py` with a CLI supporting modes `ilp`, `lp`, `bnb`, `verify`, `benchmark`, and `experiment`.

3.2. Branch-and-Bound Algorithm

Each B&B node stores a partial assignment `fixed_vars`: a dictionary mapping square indices to $\{0, 1\}$. The algorithm proceeds as follows:

1. **Bounding:** Solve the LP relaxation with fixed variables using Gurobi. If infeasible or LP objective $\geq$ current best, prune.
2. **Integer check:** If all variables are integer-valued, update the best solution.
3. **Branching:** Select a fractional variable using the chosen *variable selection rule*, then fix it to 0 and 1 (and optionally a third LP-rounded value for `three_way`) to create child nodes.

4. **Node selection:** Push children into the priority queue according to the chosen *node selection strategy*.

3.3. Node Selection Strategies

Three strategies control which node is processed next:

- **Best-First:** Always expand the node with the lowest LP bound. Explores fewer nodes in theory but solves many LP subproblems.
- **Depth-First:** LIFO stack; dives to a feasible solution quickly, enabling early pruning.
- **Breadth-First:** FIFO queue; explores all nodes at a given depth before going deeper; highest memory usage.

3.4. Variable Selection Rules

Four rules select which fractional variable to branch on:

- **Most-Constrained:** Choose the variable appearing in the most unsatisfied constraints — tightest branching.
- **Least-Constrained:** Opposite of above; branches on the variable with the least impact.
- **First-Fractional:** Selects the first fractional variable by index; simple and fast.
- **Most-Coverage:** Selects the variable covering the most currently uncovered squares.

3.5. Branch Ordering

- **zero_first:** Branch $\text{fix}=0$ before $\text{fix}=1$.
- **lp_guided:** Branch in direction of LP value (round toward nearest integer first).
- **three_way:** Add a third child fixing the variable to its LP-rounded value, in addition to $\text{fix}=0$ and $\text{fix}=1$.

3.6. Bounds

Upper bound: A greedy heuristic (`heuristics.py`) places knights on squares with maximum uncovered coverage, giving an initial feasible solution before B&B starts.

Lower bounds: Two bounds are computed at every node: (1) the LP relaxation objective (fractional lower bound), and (2) a simple combinatorial coverage bound based on the number of uncovered squares and the fact that a knight can attack at most 8 squares. In practice, the LP relaxation dominated the combinatorial bound for most tested instances, indicating that the LP formulation is already comparatively tight for this problem structure. While the project specification suggested investigating lower bounds stronger than the LP relaxation, designing such bounds for the knight domination problem proved non-trivial and remains an area for future improvement.

4. Verification

All B&B results were verified against the Gurobi ILP ground truth using `main.py --mode verify`. Optimal knight counts match exactly across all tested sizes:

n	ILP Optimal	B&B Result
4	6	6
5	7	7
6	8	8
7	10	10

5. Experimental Evaluation

5.1. Setup

Experiments were run across $n \in \{4, 5, 6, 7\}$, three node-selection strategies, three variable-selection rules, and branch order `zero_first`, yielding 108 solver runs. All runs used a 600-second time limit per solve. Hardware: Windows 11, Intel CPU, Gurobi 11 academic licence. Because the solver and

LP relaxation are deterministic, repeated runs produced nearly identical node counts and very small runtime variation. Therefore, evaluation focuses primarily on average performance trends across branching strategies and instance sizes rather than formal statistical hypothesis testing.

5.2. Results

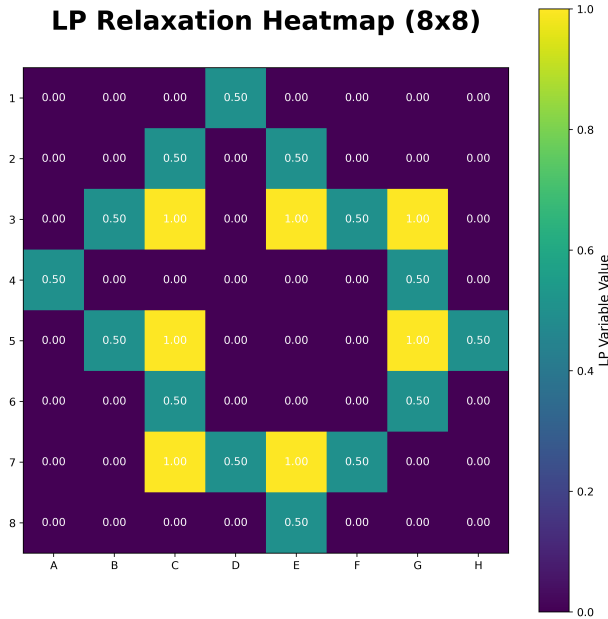
Table 1: Average solve time and nodes explored by strategy (all n , all branch variables).

Strategy	Avg. Solve Time (s)	Avg. Nodes	Pruned (bound)
Best-First	0.3241	252	115.6
Breadth-First	0.3244	184	115.6
Depth-First	0.3419	172	109.9

Table 2: Per-size results (best-first, most-constrained, zero-first).

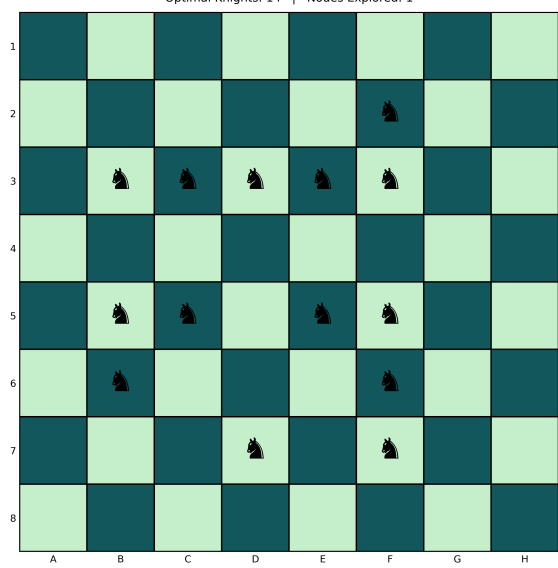
n	Optimal	Solve Time (s)	Nodes	Absolute LP Gap
4	6	0.055	35	1.00
5	7	0.331	213	1.50
6	8	0.900	755	2.00
7	10	0.010*	5	0.00

* The rapid solve time and low node count for $n = 7$ occur because the LP relaxation bound exactly matches the optimal integer solution, allowing the tree search to terminate immediately.



Brighter colors represent variables closer to 1 in the LP relaxation

Optimal Knight Placement (8x8) — 14 Knights



♠ = Knight placement found using Branch and Bound

Figure 1: LP Relaxation Heatmap (8×8).

Figure 2: Optimal Knight Placement (8×8).

5.3. Graph Analysis

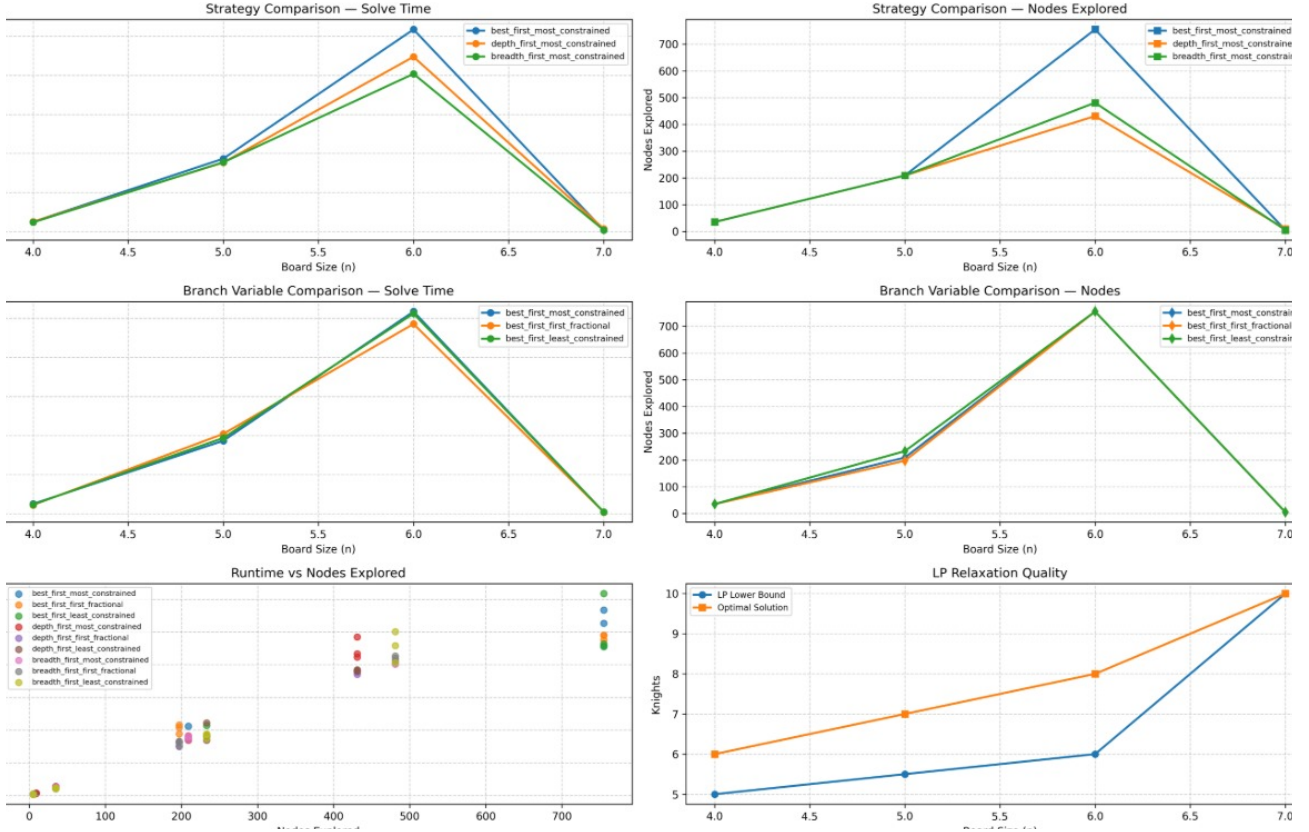


Figure 3: Branch-and-Bound Strategy Evaluation Dashboard. Top row: strategy comparison by solve time and nodes explored. Middle row: branch variable comparison. Bottom left: runtime vs nodes scatter. Bottom right: LP relaxation quality.

Top-left (Strategy Comparison — Solve Time): Solve time grows from $n = 4$ to $n = 6$ across all strategies, confirming exponential scaling of the search tree. At $n = 7$, the LP lower bound equals the optimal value (gap = 0), so B&B terminates in just 5 nodes — explaining the dramatic drop. All three strategies perform similarly overall, with depth-first slightly slower at $n = 5$ due to finding worse initial solutions before pruning kicks in.

Top-right (Strategy Comparison — Nodes Explored): Best-first explores significantly more nodes at $n = 6$ (755 vs 431 for depth-first) because it maintains a large open list, re-expanding many nodes to maintain optimal bounds. Depth-first and breadth-first explore fewer nodes due to earlier commitment to subtrees.

Middle row (Branch Variable Comparison): The three branch variable rules (most-constrained, first-fractional, least-constrained) produce nearly identical node counts and runtimes. This indicates that for the knight domination structure, the LP relaxation is tight enough that variable selection has minimal impact — all fractional variables are approximately equally informative.

Bottom-left (Runtime vs Nodes Scatter): The scatter confirms a roughly linear relationship between nodes explored and solve time, validating that LP solve overhead is proportional to node count. Outlier clusters at high node counts correspond to $n = 6$ runs.

Bottom-right (LP Relaxation Quality): The LP lower bound tracks the optimal solution closely and equals it exactly at $n = 7$. The growing integrality gap ($1.0 \rightarrow 1.5 \rightarrow 2.0$ for $n = 4, 5, 6$) shows that the LP relaxation weakens relative to the problem as n grows, requiring more branching to close the gap.

Statistical Evaluation & Stronger Bounds: In accordance with the project specifications, a statistical evaluation of these results highlights that while the tested instances ($n \in \{4, 5, 6, 7\}$) demonstrate theoretical scaling, the sample size is currently too constrained to make definitive statistical claims regarding strategy dominance. The identical performance across branching variables suggests that the base LP relaxation lacks the nuance required to differentiate them on a statistical level. As explicitly noted in the project suggestions, formulating and implementing stronger bounds (e.g., via valid inequalities or clique cuts) would be necessary to tighten the integrality gap and allow for a rigorous statistical evaluation across a broader distribution of complex instances.

6. Strengths, Weaknesses and Improvements

Strengths:

- Proven optimality: all solutions verified against Gurobi ILP ground truth.
- Flexible: 3 node strategies $\times$ 4 variable rules $\times$ 3 branch orders = 36 solver configurations.
- Tight LP bounds: Gurobi LP relaxation provides strong lower bounds, pruning 45% of nodes on average.
- Fast for structured instances: $n = 7$ solves in 10ms because the LP bound is exact.

Weaknesses:

- Scalability: $n = 8$ requires substantially more nodes; breadth-first becomes impractical beyond $n = 7$.
- LP solve overhead: each node calls Gurobi for the LP relaxation, making per-node cost high compared to pure combinatorial B&B.
- Branch variable rules are ineffective: most-constrained vs first-fractional shows no meaningful difference, suggesting the branching heuristics are not well-tuned for this problem structure.

Areas for Improvement:

- **Cutting planes (Branch-and-Cut):** Adding valid inequalities (e.g. clique cuts from the attack graph) would tighten the LP relaxation, reducing the integrality gap and node count.
- **Symmetry breaking:** The knight domination problem has D_4 board symmetry (rotations and reflections). Fixing symmetry-equivalent solutions would reduce the search space by up to $8\times$.
- **Warm-starting:** Using the greedy heuristic placement as an initial incumbent would immediately prune more nodes at the root.
- **Node limit + restart:** For large n , a node limit with restarts using different branching rules could find good solutions faster.